CortexAI Gateway Service Documentation

This document explains the setup, configuration, variables, functions, and the detailed file-to-file state flow of the Gateway service in CortexAI.

---

1. What is the Gateway? (For Beginners)

In a microservices architecture, the client's browser does not talk directly to every backend service (like Auth, Chat, or Agent). Instead, it talks to a single central server called the **Gateway** (running on port `8000`).

The Gateway acts as:

1. **A Reverse Proxy**: It receives request URLs and forwards them to the correct microservice.
2. **A Security Guard**: It intercepts requests, validates the user's session cookie against Redis, and rejects unauthorized requests.
3. **An Identity Injector**: It extracts the user's details and passes their database ID to downstream services.

---

2. Variables & Functions Dictionary

Configuration & Environment Variables

- `PORT` (defaults to `8000`): Defines the network port the Gateway server listens on.
- `FRONTEND_URL`: Points to the frontend client URL (e.g. `http://localhost:5173`) to allow cross-origin requests.
- `AUTH_SERVICE`: Target proxy URL for the Auth Service.
- `CHAT_SERVICE`: Target proxy URL for the CHAT Service.
- `AGENT_SERVICE`: Target proxy URL for the Agent Service.

Entry & Middleware Routing

- `app` (express instance in `index.js`):
  - **What it does**: The core web server application that listens on port `8000`.
- `protect` (middleware function in `auth.middleware.js`):
  - **Variables**:
    - `sessionId`: Retrieved from request cookies (`req.cookies.session`).
    - `session`: Holds the parsed user data JSON string returned from Redis.
  - **How it works**: Validates the incoming cookie against the Redis cache. If valid, it assigns the parsed object to `req.user` and calls `next()`. If invalid or missing, it blocks the request returning status `401`.

Controllers & Proxies

- `getCurrentuser` (controller in `user.controller.js`):
  - **What it does**: Handles the local `/me` route.
  - **How it works**: Returns the parsed user object from `req.user` to the client.
- `proxyWithHeader` (utility function in `proxywithheader.js`):
  - **Variables**:
    - `serviceUrl`: The destination URL of the target microservice (e.g. Chat Service or Agent Service).
    - `proxyReqOpts`: Object containing proxy request headers.
  - **How it works**: Proxies requests to downstream servers. Inside its decorator, if `srcReq.user` exists, it sets `proxyReqOpts.headers['x-user-id'] = srcReq.user.userid`.

---

3. Global Integration (Where, When, & How Gateway Variables/Functions are Used)

Gateway variables and functions are integrated across the project to protect routes and forward identities:

A. Middleware Guard Integration (`protect`)

- **Where it is used**: Imported and registered inside `gateway/index.js`.
- **When & How**: Applied as a route-level middleware to secure endpoints:
  - `app.use("/chat", protect, ...)`
  - `app.use("/agent", protect, ...)`
  - `app.get("/me", protect, ...)`
- **Result**: Any frontend request attempting to reach Chat history, run an Agent prompt, or fetch user profile data is automatically forced to pass the cookie-to-Redis validation first.

B. User Identity Forwarding (`proxyWithHeader`)

- **Where it is used**: Imported and registered inside `gateway/index.js`.
- **When & How**: Applied to proxy definitions:
  - `proxyWithHeader(process.env.CHAT_SERVICE)`
  - `proxyWithHeader(process.env.AGENT_SERVICE)`
- **Result**: When requests are forwarded to the Chat or Agent services, this utility automatically injects the `x-user-id` HTTP header.
- **Downstream Consumption**:
  - **Chat Service** (`chat.controller.js`): Reads `req.headers["x-user-id"]` to link newly created conversations or messages to the correct user.
  - **Agent Service** (`agent.controller.js`): Reads the header to identify the user before running AI logic.

C. Browser Cookie Management (`session`)

- **Where it is used**: Written by `auth.controller.js` (Auth Service) and consumed by `auth.middleware.js` (Gateway).
- **When & How**:
  - On successful login, the Auth Service sets the browser cookie `session`.
  - On subsequent requests, the browser sends this cookie, allowing the Gateway's `protect` middleware to parse it and run the Redis lookup.

---

4. File-to-File Gateway Execution Workflow

This is the exact sequence of how a protected request (like loading chat history) is processed through the Gateway files:

```
[Request Hitting Gateway] Client browser calls "/chat/get-conversation"
   |
   ▼
[Server Config] gateway/index.js
   |  ──> Matches route path "/chat".
   |  ──> Runs guard middleware protect.
   ▼
[Session Lookup] gateway/middlewares/auth.middleware.js
   |  ──> Extracts sessionId cookie.
   |  ──> Queries Redis for session data.
   |  ──> Populates req.user with { userid, name, email, avatar }.
   |  ──> Executes next() to pass control back to router.
   ▼
[Header Injection] gateway/utils/proxywithheader.js
   |  ──> Wraps proxy request.
   |  ──> Reads req.user.userid and assigns it to header key "x-user-id".
   ▼
[Proxy Dispatch] gateway/index.js
      ──> Dispatches request to Chat Service on Port 8002 with the custom headers.
```